

차트에서 한글 지원

```
import platform
from matplotlib import font_manager, rc
import matplotlib
matplotlib.rcParams['axes.unicode_minus'] = False
if platform.system() == 'Windows':
    path = "c:\\Windows\\Fonts\\malgun.ttf"
    font_name = font_manager.FontProperties(fname=path).get_name()
    rc('font', family=font_name)
elif platform.system() == 'Darwin':
    rc('font', family='AppleGothic')
elif platform.system() == 'Linux':
    rc('font', family='NanumBarunGothic')
```

구글 코랩에서

1) 한글 폰트를 설치

```
!apt-get update -qq
!apt-get install -qq fonts-nanum
```

2) 설치된 폰트를 확인

```
!fc-list :lang=ko
```

3) 세션 다시 시작 : 런타임 - 세션 다시 시작

4) matplotlib에 폰트를 설정

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import matplotlib as mpl
font_path = '/usr/share/fonts/truetype/nanum/NanumGothic.ttf'
font_name = fm.FontProperties(fname=font_path).get_name()
plt.rc('font', family=font_name)
plt.rcParams['axes.unicode_minus'] = False
```

구글 코랩에서

```
import sys

# Google Colab 환경에서 실행 중인지 확인
if 'google.colab' in sys.modules:
    # debconf를 Noninteractive 모드로 설정
    !echo 'debconf debconf/frontend select Noninteractive' | \
    debconf-set-selections

    # fonts-nanum 패키지를 설치
    !sudo apt-get -qq -y install fonts-nanum

    # Matplotlib의 폰트 매니저 가져오기
    import matplotlib.font_manager as fm

    # 나눔 폰트의 시스템 경로 찾기
    font_files =
    fm.findSystemFonts(fontpaths=['/usr/share/fonts/truetype/nanum'])

    # 찾은 각 나눔 폰트를 Matplotlib 폰트 매니저에 추가
    for fpath in font_files:
        fm.fontManager.addfont(fpath)
```

```
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'NanumGothic'
plt.rc('font', family='NanumBarunGothic', size=11)
print(plt.rcParams['font.family'], plt.rcParams['font.size'])

plt.plot([1,4,9,16])
plt.title('한글')
plt.show()
```

시각화 패키지 Matplotlib

```
import matplotlib.pyplot as plt
plt.figure(figsize=(3,2))
```

라인 플롯

```
plt.title("x축의 tick 위치를 명시")
plt.plot([10,20,30,40],[1,4,9,16])
plt.show()  📖 실제로 차트로 렌더링(rendering)
```

스타일 지정

스타일 문자열은 색깔(color), 마커(marker), 선 종류(line style)의 순서로 지정

```
plt.title("'rs--' 스타일의 plot ")
plt.plot([10,20,30,40],[1,4,9,16],'rs--')
```

색

색이름, 약자, #RGB코드 사용한다.

blue	b	green	g	red	r	cyan	c
magenta	m	yellow	y	black	k	white	w

마커 : 데이터 위치를 나타내는 기호

.	point marker	,	pixel marker
o	circle marker	v	triangle_down marker
^	triangle_up marker	<	triangle_left marker
>	triangle_right marker	1	tri_down marker
2	tri_up marker	3	tri_left marker
4	tri_right marker	s	square marker
p	pentagon marker	*	star marker
h	hexagon1 marker	H	hexagon2 marker
+	plus marker	x	x marker
D	diamond marker	d	thin_diamond marker

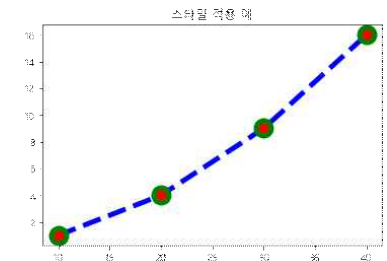
선 스타일

-	solid line style	--	dashed line style
-.	dash-dot line style	:	dotted line style

기타 스타일 - 라인 플롯에서 자주 사용되는 기타 스타일

color	c	선 색깔
linewidth	lw	선 굵기
linestyle	ls	선 스타일
marker		마커 종류
markersize	ms	마커 크기
markeredgewidth	mew	마커 선 굵기
markerfacecolor	mfc	마커 내부 색깔

```
plt.plot([10,20,30,40],[1,4,9,16], c="b",
lw=5, ls="--", marker="o", ms=15,
mec="g", mew=5, mfc="r")
plt.title("스타일 적용 예")
```

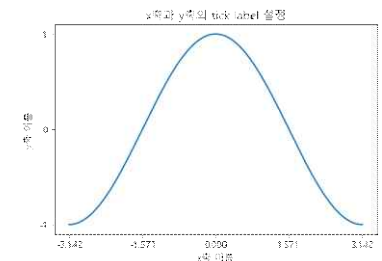


X,Y 범위 xlim, ylim 명령

```
plt.title("x축, y축의 범위 설정")
plt.plot([10,20,30,40],[1,4,9,16], c="b", lw=5, ls="--",
marker="o", ms=15, mec="g", mew=5, mfc="r")
plt.xlim(0,50)  📖 x축의 최소, 최대값
plt.ylim(-10,30)  📖 y축의 최소, 최대값
```

틱 xticks, yticks 명령

```
X=np.linspace(-np.pi,np.pi,256)
C=np.cos(X)
plt.title("x축과 y축의 tick label 설정")
plt.plot(X,C)
plt.xlabel("x축 이름")
plt.ylabel("y축 이름")
plt.xticks([-np.pi,-np.pi/2,0,np.pi/2,np.pi])
plt.yticks([-1,0,+1])
```



범례 legend 명령

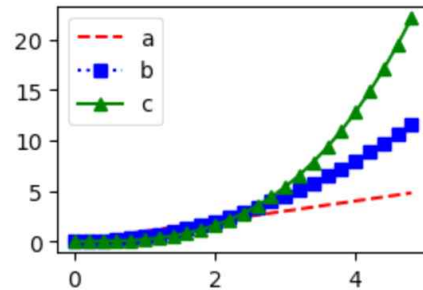
범례의 위치 : 수동으로 설정하고 싶으면 loc 인수를 사용

loc문자열	숫자	lower left	3	center right	7
best	0	lower right	4	lower center	8
upper right	1	right	5	upper center	9
upper left	2	center left	6	center	10

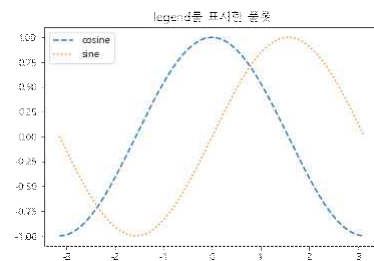
여러개의 선을 그리기

```
t=np.arange(0.,5.,0.2)
plt.title("라인 플롯 여러개 선 그리기")
plt.plot(t,t,'r--',t,0.5*t**2,
         'bs:',t,0.2*t**3,'g^-')

plt.plot(t,t,'r--',          label="a")
plt.plot(t,0.5*t**2,'bs:',   label="b")
plt.plot(t,0.2*t**3,'g^-',   label="c")
plt.legend(loc=0)
```



```
X=np.linspace(-np.pi,np.pi,256)
C,S=np.cos(X),np.sin(X)
plt.title("legend를 표시한 플롯")
plt.plot(X,C,ls="--",label="cosine")
plt.plot(X,S,ls=":",label="sine")
plt.legend(loc=2)
```



히스토그램

```
x = np.random.randint(1,7,1000)
plt.hist(x, bins = 6, width=0.5)
```

원그래프

```
times = [8, 14, 2]
timelabels = ["Sleep", "Study", "Play"]
plt.pie(times, labels = timelabels, autopct = "%.2f")
```

2x2 형태의 네 개의 플롯

```
np.random.seed(0)

plt.subplot(221)
plt.plot(np.random.rand(5))
plt.title("axes 1")

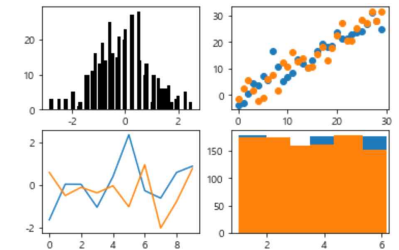
plt.subplot(222)
plt.plot(np.random.rand(5))
plt.title("axes 2")
```

```
plt.subplot(223)
plt.plot(np.random.rand(5))
plt.title("axes 3")

plt.subplot(224)
plt.plot(np.random.rand(5))
plt.title("axes 4")

plt.tight_layout()
plt.show()
```

```
fig = plt.figure()
ax3 = fig.add_subplot(2, 2, 3)
ax1 = fig.add_subplot(2, 2, 1)
ax2 = fig.add_subplot(2, 2, 2)
ax4 = fig.add_subplot(2, 2, 4)
```



```
ax1.hist(np.random.randn(200), bins=20, color='k', rwidth=0.5)
ax2.scatter(np.arange(30), np.arange(30) + 3*np.random.randn(30))
ax3.plot(np.arange(10),np.random.randn(10))
ax4.hist(np.random.randint(1,7,1000), bins = 6, width=1)
fig
```

```
df = DataFrame(np.random.rand(6, 4),
               index=['one', 'two', 'three', 'four', 'five', 'six'],
               columns=pd.Index(['A', 'B', 'C', 'D'], name='Genus'))
df.plot(kind='bar')
```

```
df.plot(kind='barh', stacked=True)
```